

FOC Debug Extraction Data Guide

Power Application Controller®





Table of Contents

1 Features 3

2 How to use the debug data extraction in FOC firmware 3

3 Capture data with fixed variables. 3

4 Capture data with customized variables. 5

5 Auto-trigger condition 5

Contact Information..... 7

Important Notice 7

1 FEATURES

- Support debugging FOC application with monitoring linear, non-linear, or non-scalable variables.
- Configurable data in various ways: continuous, skip-cycles.
- GUI automatically limit the maximum sampling cycles from available target RAM size.
- Retry and timeout in reading result.
- Save data into the csv file for further analyses.
- The function time adds to control loop time:
 - 1 Parameter: 1.45us added.
 - 4 Parameters: 2.1us added.

2 HOW TO USE THE DEBUG DATA EXTRACTION IN FOC FIRMWARE

To use debug data extraction in FOC firmware we need to enable the configuration in config_debug.h.

- 1) Uncomment ENABLE_DEBUG and ENABLE_DEBUG_SAMPLE_RAM

```
#define ENABLE_DEBUG //
#ifdef ENABLE_DEBUG
// #define ENABLE_DEBUG_PWM DAC ,
#define ENABLE_DEBUG_DISPLAY //
#define ENABLE_DEBUG_IO
#define ENABLE_DEBUG_INPUT
// #define ENABLE_DEBUG_DAS
#define ENABLE_DEBUG_PARAMEXPORT //
#define ENABLE_FOCCONTROLTIME_OVERFLOW_CHECK //
#define ENABLE_STACK_CHECK FALSE
// #define IPD_DEBUG_ENABLE
// #define ENABLE_DEBUG_VOLTAGEMODE
// #define ENABLE_TEMPREAD
#define ENABLE_DEBUG_SAMPLE_RAM
```

- 2) Configure the size of buffer array.

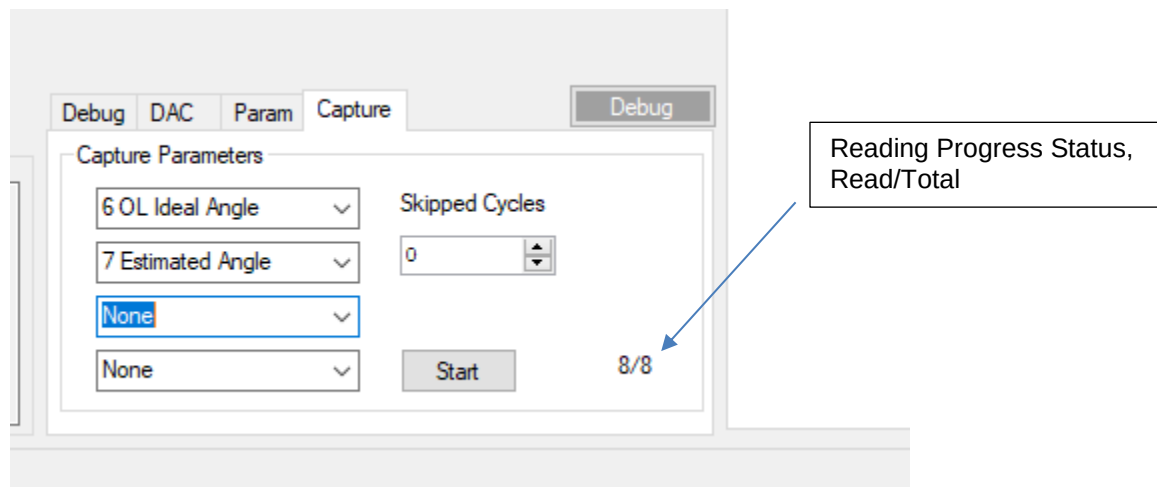
Please note that this feature based on RAM side, the RAM memory will be consumed is **DBG_ARRAY_LENGTH x 4** (bytes). More buffer array length, more cycles & variables could be captured.

```
#if defined(ENABLE_DEBUG_SAMPLE_RAM)
// #warning "Debug extraction data to RAM was enabled, please consider RAM size
#define DBG_ARRAY_LENGTH (480) // 480x4 = 1920 bytes
#endif
```

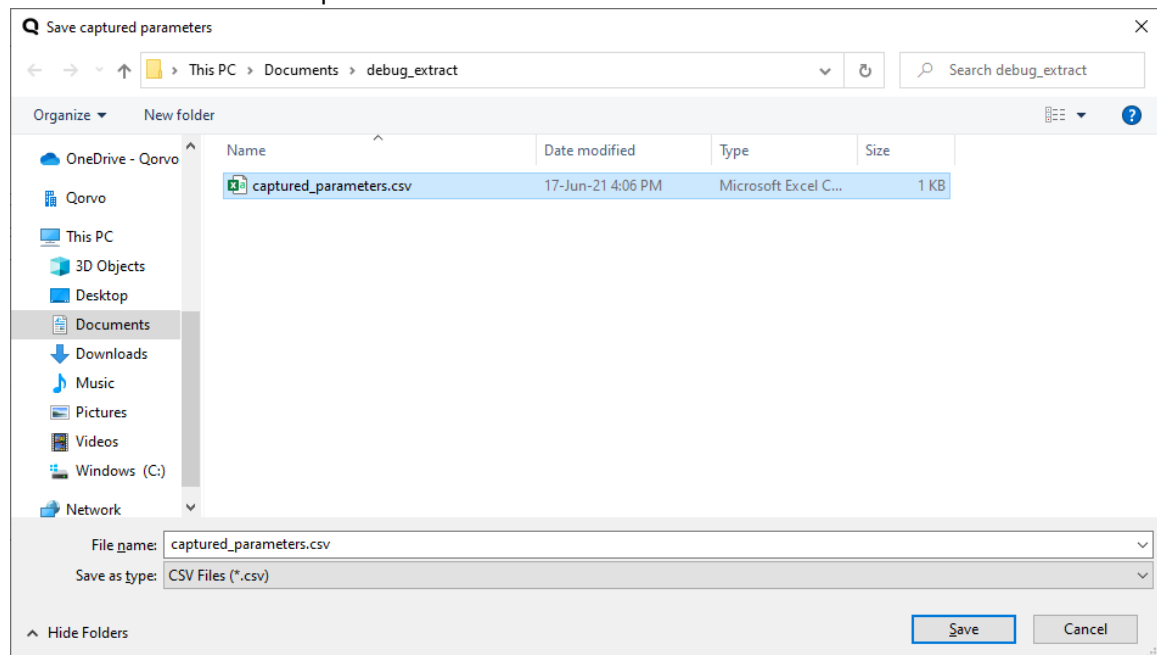
3 CAPTURE DATA WITH FIXED VARIABLES.

- Click on Capture tab of debug feature.
- Select parameters will be captured, "None" means unselected.
- Select number of skipped cycles, "0" means continuous mode.
E.g., skipped cycles = 1 the data will be captured in control loop follow the sequence

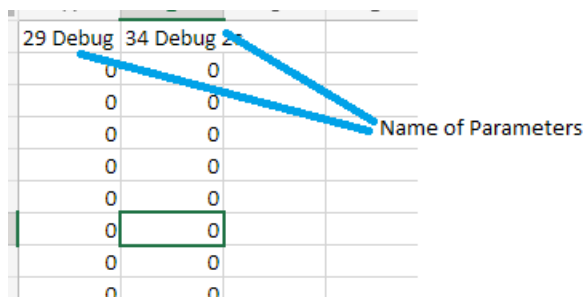
capture – skip – capture.



- Click on “Start” button to kick off the sequence, we must wait until it ends.
- Save data file when it completes.



- Output CSV Format



A screenshot of a debug data table. The table has two columns labeled '29 Debug' and '34 Debug'. The rows contain numerical values (0, 0, 0, 0, 0, 0, 0, 0, 0, 0). A blue arrow points from the text 'Name of Parameters' to the header '29 Debug'. Another blue arrow points from the same text to the header '34 Debug'.

29 Debug	34 Debug
0	0
0	0
0	0
0	0
0	0
0	0
0	0
0	0
0	0
0	0

4 CAPTURE DATA WITH CUSTOMIZED VARIABLES.

User can define exact data should be captured by debug-variables in firmware:

Here is the list of debug variables could be used in FOC firmware.

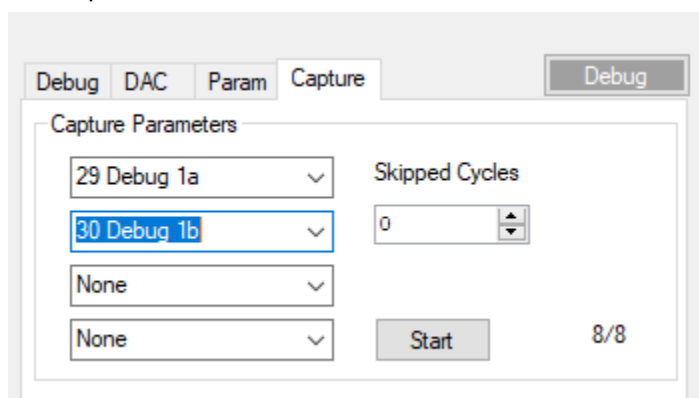
1	debug_dac_1a
2	debug_dac_1b
3	debug_dac_1c
4	debug_dac_2a
5	debug_dac_2b
6	debug_dac_2c

Step 1: Assign debug-variables with intended parameters to debug in firmware source code and re-compile. E.g.

```
motor_ptr->threeshunt.Iu_ADC = Iu_temp;
motor_ptr->threeshunt.Iv_ADC = Iv_temp;
motor_ptr->threeshunt.Iw_ADC = Iw_temp;
```

```
debug_dac_1a = motor_ptr->threeshunt.Iu_ADC;
debug_dac_1b = motor_ptr->threeshunt.Iv_ADC;
```

Step 2: Select parameter on GUI and start.



A screenshot of the GUI showing the 'Capture Parameters' section. The 'Debug' tab is selected. The 'Capture Parameters' section has three dropdown menus. The first dropdown is set to '29 Debug 1a', the second to '30 Debug 1b', and the third to 'None'. To the right of the dropdowns is a 'Skipped Cycles' field with a value of '0'. At the bottom right is a 'Start' button and a counter '8/8'.

5 AUTO-TRIGGER CONDITION

Normally, the data extraction should be captured manually by GUI but in some specific cases, it may be "triggered" by the firmware itself.

User must enable start-condition by calling to API in firmware code. After that, the sequence will run automatically until the end cycle.

E.g.

```
void debug_extraction_start(Struct_Motor* motor_ptr)
```

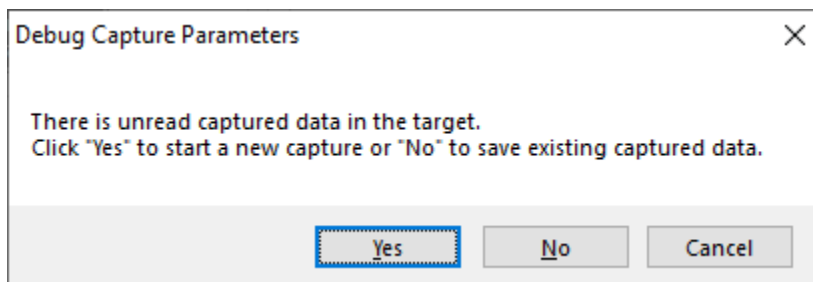
When motor is in start-up state

```
switch(motor_ptr->startup.aligngo.state_control)
{
    case STATE_ALIGNGO_CONTROL_ALIGN:
        // Increment counter to keep track of align time
        motor_ptr->startup.aligngo.aligntime_counter++;
#ifdef ENABLE_OPDOL
        statemachine_opdol_ctl(motor_ptr);
#endif
        debug_extraction_start(motor_ptr);
        break;
```

You, 4 minutes

Step 1: User must configure parameters which to be captured by GUI before enabling motor for auto-trigger.

Step 2: Click the “Start” button and check whether data is captured by FOC firmware or not?





Debug Extraction Data Guide

Power Application Controller®

CONTACT INFORMATION

For the latest specifications, additional product information, worldwide sales and distribution locations:

Web: www.qorvo.com

Tel: 1-844-890-8163

Email: customer.support@qorvo.com

IMPORTANT NOTICE

The information contained herein is believed to be reliable; however, Qorvo makes no warranties regarding the information contained herein and assumes no responsibility or liability whatsoever for the use of the information contained herein. All information contained herein is subject to change without notice. Customers should obtain and verify the latest relevant information before placing orders for Qorvo products. The information contained herein or any use of such information does not grant, explicitly or implicitly, to any party any patent rights, licenses, or any other intellectual property rights, whether with regard to such information itself or anything described by such information. **THIS INFORMATION DOES NOT CONSTITUTE A WARRANTY WITH RESPECT TO THE PRODUCTS DESCRIBED HEREIN, AND QORVO HEREBY DISCLAIMS ANY AND ALL WARRANTIES WITH RESPECT TO SUCH PRODUCTS WHETHER EXPRESS OR IMPLIED BY LAW, COURSE OF DEALING, COURSE OF PERFORMANCE, USAGE OF TRADE OR OTHERWISE, INCLUDING THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.**

Without limiting the generality of the foregoing, Qorvo products are not warranted or authorized for use as critical components in medical, life-saving, or life-sustaining applications, or other applications where a failure would reasonably be expected to cause severe personal injury or death.

Copyright 2019 © Qorvo, Inc. | Qorvo is a registered trademark of Qorvo, Inc.